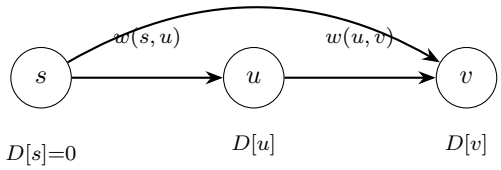
1. Checking Correctness

Algorithm



Return **true** iff all three conditions hold:

1. $D[s] = 0$
2. $\forall\,(u, v) \in E$: $\quad D[v] \le D[u] + w(u, v)$ (triangle inequality)
3. $\forall\, v \neq s$ with $D[v] < \infty$: $\quad \exists\,(u, v) \in E$ with $D[v] = D[u] + w(u, v)$ (tightness)

Proof of correctness

Necessary: Shortest distances satisfy (1)–(3) by definition.

Sufficient ($D[v] = \text{dist}(s, v)$ when (1)–(3) hold):

Upper bound: For any path $s = u_0, u_1, \ldots, u_k = v$, applying (2) inductively:

$$D[v] \le D[u_{k-1}] + w(u_{k-1}, v) \le \cdots \le \sum_{i=0}^{k-1} w(u_i, u_{i+1})$$

$\implies D[v] \le \text{dist}(s, v)$.

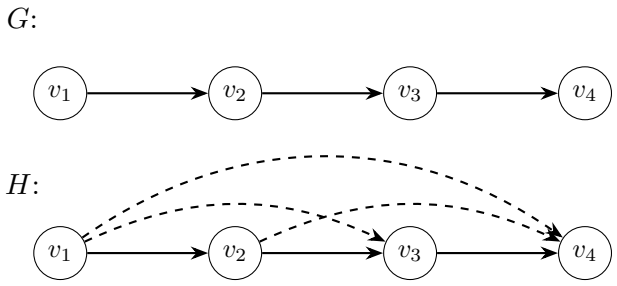
Lower bound: For v with $D[v] < \infty$, condition (3) gives a predecessor $\pi(v)$ with $D[v] = D[\pi(v)] + w(\pi(v), v)$. Since $w \ge 0$, $D[\pi(v)] \le D[v]$, so the chain $v, \pi(v), \pi^2(v), \ldots$ has non-increasing D-values and visits at most n distinct vertices before reaching s (where $D[s] = 0$). This chain yields a path from s to v of weight $D[v]$, so $\text{dist}(s, v) \le D[v]$.

Unreachable vertices: If $D[v] = \infty$ but v is reachable, the upper bound gives $D[v] \le \text{dist}(s, v) < \infty$, a contradiction. So $D[v] = \infty \iff v$ is unreachable. □

Runtime: Conditions (1)–(3) each require a single scan of vertices/edges.

$$\boxed{O(m + n)}$$

2. Transitive Closure



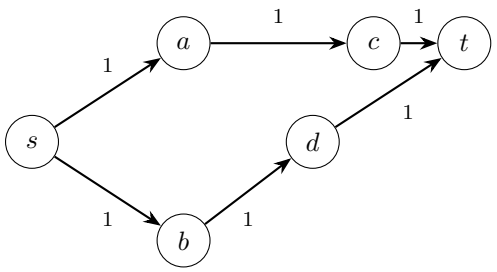
begin
  TransitiveClosure($G = (V, E)$)
  Compute a topological ordering $v_1, v_2, \ldots, v_n$ of G
  for $i \longleftarrow n$ ***down to*** 1 **do**
    Reach[v_i] $\longleftarrow$ n-bit vector with bit v_i set
    for ***each*** $(v_i, v_j) \in E$ **do**
      Reach[v_i] $\longleftarrow$ Reach[v_i] | Reach[v_j]
  return $H = \{(u, v) : \textit{bit } v \textit{ is set in } \text{Reach}[u]\}$

Correctness: Processing in reverse topological order ensures Reach[v_j] is complete before any predecessor v_i reads it (since $i < j$ for every edge (v_i, v_j)). After processing v_i, Reach[v_i] contains all vertices reachable from v_i.

Runtime: Topological sort $O(m + n)$. Each of m edges triggers one OR of n-bit vectors ($O(n)$ each).

$$\boxed{O(mn)}$$

3. Fault-Tolerant Paths



All capacities = 1; two edge-disjoint s-t paths

Solution 1:

Reduction: Assign capacity 1 to every edge. By Menger's theorem, the maximum number of edge-disjoint s-t paths equals the max flow from s to t in this unit-capacity network.

1. Build unit-capacity network on G.
2. Run Edmonds-Karp (BFS-based Ford-Fulkerson). Each augmenting path finds one unit of flow.
3. If max flow ≥ 2: decompose the flow into 2 paths by tracing from s along edges with flow $= 1$ to t, removing used edges after each trace.
4. If max flow < 2: report no two edge-disjoint s-t paths exist.

Runtime: ≤ 2 BFS iterations, each $O(m)$. Flow decomposition: $O(m)$. $\boxed{O(m)}$

Solution 2:

Reduction: Assign capacity 1 to every edge.

1. Build unit-capacity network on G. Initialize flow $f(e) = 0$ for all e.
2. **Repeat up to 2 times:** Run DFS on the residual graph to find an augmenting s-t path. If found, push 1 unit of flow along the path and update the residual graph. If no path exists, stop.
3. If total flow ≥ 2: decompose into 2 edge-disjoint paths. Else report no two edge-disjoint s-t paths exist.

Decomposition: Trace from s along edges with $f(e) = 1$ until t; remove used edges; repeat.

Correctness:

($\Rightarrow$) $\exists$ two edge-disjoint s-t paths $\implies$ send 1 unit along each $\implies$ feasible flow of value 2 $\implies$ max flow ≥ 2.

($\Leftarrow$) Max flow ≥ 2 $\implies$ $\exists$ integer flow with $\forall e$: $f(e) \in \{0, 1\}$ (integrality theorem). Flow conservation $\implies$ tracing from s along f=1 edges reaches t. Remove first path; remaining flow has value 1; repeat $\implies$ two edge-disjoint paths.

Runtime: ≤ 2 DFS iterations, each $O(m+n)$. Decomposition: $O(m)$. $\boxed{O(m+n)}$

(a)

Yes. The same reduction generalizes: k edge-disjoint s-t paths exist $\iff$ max flow $\geq k$ in the unit-capacity network. Run Ford-Fulkerson for up to k augmenting path iterations.

(b)

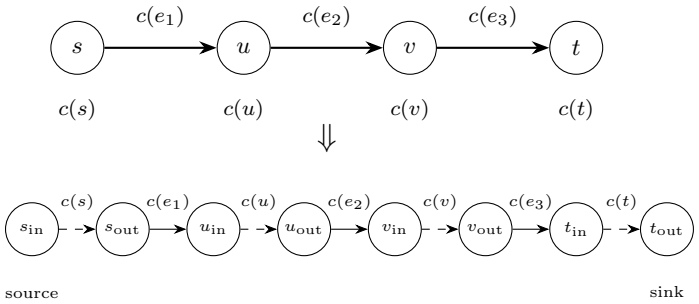
k DFS augmenting path iterations $\times$ $O(m+n)$ each + $O(km)$ decomposition.

$$\boxed{O(k(m+n))}$$

4. Vertex Capacities

Reduction: Split each vertex v into v_{in} and v_{out}:

- All incoming edges of v redirect to v_{in}
- All outgoing edges of v redirect from v_{out}
- Add internal edge $(v_{\text{in}}, v_{\text{out}})$ with capacity $c(v)$
- Original edge (u, v) with capacity $c(e)$ becomes $(u_{\text{out}}, v_{\text{in}})$ with capacity $c(e)$



New source $= s_{\text{in}}$, new sink $= t_{\text{out}}$. The new graph G' has $2n$ vertices and $m+n$ edges.

Correctness:

($\Rightarrow$) Given a feasible flow f' in G', define $f(u, v) = f'(u_{\text{out}}, v_{\text{in}})$ in G. Edge capacities are preserved. The total flow through v equals $f'(v_{\text{in}}, v_{\text{out}}) \leq c(v)$, so the vertex capacity is satisfied.

($\Leftarrow$) Given a feasible flow f in G with vertex capacities, define:

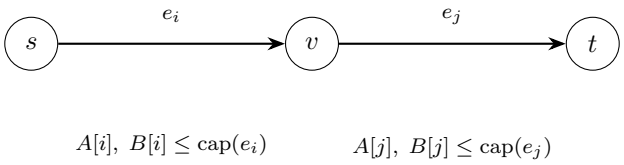
$$f'(u_{\text{out}}, v_{\text{in}}) = f(u, v), \qquad f'(v_{\text{in}}, v_{\text{out}}) = \textstyle\sum_u f(u, v)$$

Since f respects $c(v)$: $f'(v_{\text{in}}, v_{\text{out}}) \leq c(v)$. Conservation in G' follows from conservation in G.

$\implies$ Max flow in G' = max flow in G with vertex capacities. Run any max-flow algorithm on G'. $\square$

Reduce to standard max-flow on G' with $2n$ vertices, $m + n$ edges

5. Convex Combination of Flows



Let f_1, f_2 be feasible flows with arrays $A[1..m]$, $B[1..m]$. Define $C[i] = c \cdot A[i] + (1 - c) \cdot B[i]$ for $0 \leq c \leq 1$.

Capacity constraints: Since $c \geq 0$, $1 - c \geq 0$, $A[i] \geq 0$, $B[i] \geq 0$:

$$C[i] = c \cdot A[i] + (1 - c) \cdot B[i] \geq 0$$

Since $A[i] \leq \text{cap}(e_i)$ and $B[i] \leq \text{cap}(e_i)$:

$$C[i] \leq c \cdot \text{cap}(e_i) + (1 - c) \cdot \text{cap}(e_i) = \text{cap}(e_i)$$

Flow conservation: For each vertex $v \neq s, t$:

$$\begin{aligned}\sum_{\text{in}} C[i] - \sum_{\text{out}} C[i] &= c\left(\sum_{\text{in}} A[i] - \sum_{\text{out}} A[i]\right) + (1 - c)\left(\sum_{\text{in}} B[i] - \sum_{\text{out}} B[i]\right) \\ &= c \cdot 0 + (1 - c) \cdot 0 = 0\end{aligned}$$

$\implies$ $C[1..m]$ satisfies capacity constraints and flow conservation $\implies$ C is a feasible flow. $\square$

$$\boxed{|f_C| = c \cdot |f_1| + (1 - c) \cdot |f_2|}$$